ЛАБОРАТОРНАЯ РАБОТА № 2

Чтение информации из графических файлов

Цель работы

Закрепить теоретические знания о форматах хранения растровых изображений и освоить способы получения основной информации об изображении из бинарной структуры файла. Разработать веб-приложение для чтения метаданных отдельных изображений и файлов выбранной папки.

Задачи работы

- Изучить структуру форматов JPG, GIF, TIF, BMP, PNG и PCX.
- Реализовать получение имени файла, размеров в пикселях, разрешения DPI, глубины цвета и способа сжатия.
- Предусмотреть выбор папки с вложенными каталогами и ограничением до 100 000 файлов.
- Разделить приложение на Presentation Layer, Core / Business Logic Layer и Data / Parsing Layer.
- Организовать фоновую параллельную обработку и удобное отображение результатов.
- Предусмотреть обработку ошибок отдельных файлов и проверку корректности метаданных.

Использованные средства разработки

- React — построение интерфейса и отображение результатов.
- TypeScript — типизация метаданных, заданий и взаимодействия слоёв.
- Vite — сервер разработки и подготовка production-сборки.
- HTML и CSS — разметка и оформление интерфейса.
- File API, Blob, DataView — чтение частей файлов и интерпретация бинарных полей.
- Web Workers — выполнение разбора файлов в фоновых потоках.
- Git и GitHub — средства контроля версий; для работы создана локальная ветка lab2-image-inspector.

Ход выполнения работы

1. Изучение растровых форматов

Растровое изображение представляет собой прямоугольную сетку пикселей. Графический файл содержит не только данные пикселей, но и служебные структуры: заголовки, таблицы, блоки и теги. По этим структурам можно определить основные характеристики изображения без полного декодирования растра.

Формат	Структуры, используемые при разборе
JPEG	Маркеры SOF, APP0 JFIF, APP1 Exif, DQT.
PNG	Сигнатура и блоки IHDR, pHYs, PLTE, tRNS.
GIF	Logical Screen Descriptor и первый Image Descriptor.
TIFF	Заголовок и теги первого каталога IFD.
BMP	Заголовок файла и заголовок DIB.
PCX	Фиксированный 128-байтный заголовок.

2. Определение основных характеристик

Размер изображения — ширина и высота растра в пикселях. Эти значения не определяют физический размер отпечатка без сведений о разрешении. Для GIF выводится размер логического холста, для TIFF — первой страницы. Поворот по Exif Orientation не применяется к значениям ширины и высоты.

Разрешение DPI характеризует физическую плотность изображения по горизонтали и вертикали. Если файл хранит плотность в пикселях на метр, выполняется пересчёт: DPI = px/m × 0,0254. Для пикселей на сантиметр: DPI = px/cm × 2,54. Если абсолютная единица измерения или сами значения отсутствуют, приложение не подставляет произвольные 72 или 96 DPI.

Глубина цвета показывает количество бит на пиксель в представлении формата. Для компонентных изображений она определяется по битам на компонент и числу компонентов. В индексных PNG и GIF это число бит индекса палитры, а не 24-битная глубина RGB-записи палитры.

Сжатие определяется по соответствующему полю заголовка либо правилам формата. PNG использует Deflate, GIF — LZW; TIFF и BMP допускают несколько способов хранения. Для JPEG дополнительно учитывается тип маркера SOF.

3. Организация программы

Приложение разделено на три независимых по ответственности слоя. Общие типы данных и контракты расположены в Core. Реализации парсеров не используют React, а интерфейс не интерпретирует бинарные поля графических форматов.

Presentation Layer — интерфейс

В каталоге src/presentation находятся App.tsx и styles.css. Слой отвечает за выбор файлов и папки, отображение таблицы, прогресса и подробных сведений. Через состояние React интерфейс получает результаты обработки и обновляет представление. При выборе строки может отображаться предпросмотр изображения.

Core / Business Logic Layer — менеджер обработки

В src/core/types.ts определены Metadata, Result, Job, Progress, а также контракты InspectorPort и BatchEngine. Функция collectJobs подготавливает очередь заданий. WorkerEngine управляет пулом обработчиков: выдаёт задания свободным workers, принимает результаты, отслеживает прогресс и завершает обработку по запросу пользователя.

При выборе папки фактическое перечисление её содержимого выполняет браузер. Приложение получает FileList с относительными путями webkitRelativePath, в том числе для вложенных каталогов. Поэтому браузерная версия не выполняет прямой обход диска по абсолютному пути.

Data / Parsing Layer — чтение и разбор

Каталог src/infrastructure содержит Reader, BinaryInspector, адаптер parser.worker.ts и отдельные парсеры JPEG, PNG, GIF, TIFF, BMP и PCX. Reader читает нужные участки через Blob.slice(). BinaryInspector определяет формат по сигнатуре и вызывает соответствующий парсер. Адаптер worker связывает задания Engine с реализацией InspectorPort.

Связывание слоёв

В composition.ts создаётся фабрика Web Workers и передаётся менеджеру обработки. Core не импортирует конкретные парсеры. Infrastructure использует контракты Core, а Presentation обращается к Engine. Извлечение метаданных реализовано самостоятельно: внешняя библиотека для разбора изображений не применяется.

4. Реализация побайтового чтения

Формат файла определяется по содержимому, а не только по расширению имени. Для чтения чисел применяется DataView с учётом порядка байтов. Перед чтением проверяется, что смещение и длина не выходят за границы файла или вложенного контейнера.

JPEG

Размеры, точность и число компонентов берутся из SOF. Глубина вычисляется как точность × число компонентов. DPI читается из APP0 JFIF либо APP1 Exif. При наличии физических значений Exif они имеют приоритет над JFIF; при расхождении выводится предупреждение. Маркер SOF позволяет выделить progressive и lossless-варианты JPEG. Сканирование завершается при первом SOS, без чтения сжатого потока пикселей.

PNG

Из IHDR извлекаются ширина, высота, bit depth и color type. По цветовому типу определяется число компонентов. Разрешение берётся из pHYs, если указана физическая единица — метр. Блоки обходятся по их длинам; содержимое больших IDAT не декодируется. PLTE позволяет определить размер палитры.

GIF

Из Logical Screen Descriptor читаются размеры холста и параметры глобальной палитры. Первый Image Descriptor определяет наличие локальной палитры кадра. Глубина индекса определяется по палитре первого кадра. Поле Color Resolution не принимается за глубину индексного пикселя. Стандартного физического DPI в GIF нет.

TIFF

Поддерживаются Classic TIFF и BigTIFF с порядком байтов II или MM. Разбирается первый IFD: ImageWidth, ImageLength, BitsPerSample, SamplesPerPixel, Compression, XResolution, YResolution и ResolutionUnit. Глубина определяется суммой битов компонентов. Обрабатывается первая страница, без декодирования strips или tiles.

BMP и PCX

В BMP читаются ширина, высота, biBitCount, biCompression и плотность в пикселях на метр. Для отрицательной высоты выводится её абсолютное значение, а порядок строк показывается отдельно. В PCX размеры вычисляются как Xmax − Xmin + 1 и Ymax − Ymin + 1; глубина равна BitsPerPixel × NPlanes. Разрешение берётся из HRes/VRes, способ кодирования — из Encoding.

5. Разработка интерфейса

Основной результат представлен в таблице. Для каждого успешно прочитанного файла выводятся имя, формат, размеры в пикселях, DPI X × Y, глубина в bpp и сжатие. Неизвестные значения не заменяются фиктивными. Для ошибочного файла сохраняется запись с причиной ошибки.

Базовая версия также содержит поиск, фильтры, сортировку, экспорт CSV и подробную панель. Эти возможности дополняют обязательные требования. Удаление вспомогательных счётчиков, фильтров или выбора числа потоков не изменяет сам механизм бинарного разбора.

6. Фоновая обработка и быстродействие

Для параллельного выполнения используются Web Workers. Основной поток управляет интерфейсом, фоновые обработчики читают и разбирают разные файлы. Свободный worker получает следующее задание из очереди. Один файл обрабатывается одним worker; выбранное число потоков относится ко всему пулу, а не к каждому изображению.

Reader использует небольшой кэш: при необходимости читает срез размером не менее 16 КБ, если позволяет размер файла. Вместо загрузки большого изображения целиком парсер запрашивает заголовки и нужные блоки. Это особенно полезно для JPEG, где основные сведения обычно находятся перед сжатым потоком.

Результаты передаются интерфейсу партиями с интервалом около 100 мс. При отображении большой таблицы применяется виртуализация: создаются только видимые строки и небольшой запас по краям. Высота виртуального полотна соответствует общему числу результатов.

Количество файлов ограничено 100 000. Это программно установленный предел, а не подтверждение проведённого нагрузочного теста такого масштаба. Список файлов и метаданные сохраняются в оперативной памяти, поэтому её расход растёт с количеством записей. Сортировка и фильтрация также требуют времени.

7. Дополнительные характеристики

Для JPEG извлекаются таблицы квантования DQT: 64 коэффициента выводятся в порядке zigzag, как записаны в сегменте. Эти коэффициенты используются при квантовании DCT-коэффициентов. Универсальный процент «качества JPEG» не вычисляется. Для палитровых форматов показывается размер таблицы цветов — он не равен числу реально использованных цветов изображения.

Технические сложности и способы решения

1. Различия бинарной структуры форматов

Одинаковые характеристики хранятся в разных местах и имеют разные типы данных. Решение: выделены отдельные парсеры и единый тип Metadata. Интерфейс получает одинаковую структуру результата независимо от исходного формата.

2. Неоднозначность разрешения и глубины

Файл может не содержать физического DPI, а поле глубины иногда относится к компоненту или индексу палитры. Решение: введены явные формулы пересчёта, учёт единиц и цветового типа. Отсутствующий DPI отображается как неизвестный. Для GIF не используется выдуманное физическое разрешение.

3. Большие файлы и отзывчивость интерфейса

Чтение всех пикселей и создание десятков тысяч DOM-строк увеличивают нагрузку. Решение: чтение ограниченных срезов, пропуск сжатых блоков, пул Web Workers, пакетное обновление результатов и виртуализация таблицы.

4. Повреждённые файлы и неподдерживаемые варианты

Некорректные размеры блоков и смещения могут привести к ошибкам чтения. Решение: проверка границ, ограничения числа блоков и перехват исключений для каждого задания. Неизвестный тип или неподдерживаемая разновидность заголовка выводятся как ошибка, не останавливая всю очередь.

Проверка реализации

При подготовке проекта выполнен 31 автоматический тест парсеров и менеджера обработки: 31 пройден, ошибок нет. Проверены шесть форматов, progressive JPEG, Exif/JFIF, PNG с разной глубиной, TIFF LZW, BigTIFF, big-endian TIFF, обрезанные данные, некорректные смещения, выдача заданий, отмена и ограничение числа workers.

В среде подготовки тесты запускались через Node.js после преобразования TypeScript в JavaScript средствами Babel. Проверка типов tsc, production-сборка Vite и визуальные браузерные тесты там не выполнялись из-за отсутствия необходимых npm-зависимостей. На компьютере пользователя полный цикл проверки выполняется командами npm install, npm test и npm run build.

Результаты работы

Разработано веб-приложение Raster Inspector на React и TypeScript. Реализовано чтение метаданных JPG, GIF, TIF, BMP, PNG и PCX без внешней библиотеки разбора изображений. Приложение получает сведения из бинарных заголовков и служебных структур файлов.

- Предусмотрены выбор нескольких файлов и выбор папки с вложенными каталогами.
- Выводятся имя, размеры изображения, DPI по двум осям, глубина и сжатие.
- Формат определяется по сигнатуре, а не только по расширению.
- Организованы фоновая обработка, очередь заданий и отображение прогресса.
- Реализованы раздельная обработка ошибок и виртуализация строк.
- Сохранено разделение UI, менеджера обработки и бинарных парсеров.

Пример полученного результата

На предоставленном снимке интерфейса для файла WallArtTheoryCo - Etsy.jpg отображены следующие сведения:

Характеристика	Значение
Формат	JPEG
Размер изображения	600 × 600 пикселей
Разрешение	72 × 72 DPI
Глубина	24 bpp
Сжатие	JPEG DCT, с потерями, progressive

Эти значения иллюстрируют результат работы интерфейса для одного файла; независимая сверка исходного изображения не выполнялась. Архив преподавателя «Для проверки Lab#2» и нагрузка около 600 JPEG общим объёмом 2 ГБ не предоставлялись для проверки при подготовке проекта. Публичное размещение веб-приложения в рамках подготовки архива не выполнялось.

Вывод

В ходе лабораторной работы рассмотрены способы хранения метаданных в основных растровых форматах и реализовано их побайтовое чтение. Установлено, что корректное определение DPI и глубины требует учитывать не только числовое значение, но и единицы измерения, цветовой тип, число компонентов и наличие палитры.

Разделение программы на три слоя позволило отделить представление результатов от организации обработки и особенностей форматов. Чтение срезов, Web Workers и виртуальная таблица уменьшают объём лишней работы и создают основу для обработки больших каталогов. Окончательную оценку корректности и быстродействия необходимо проводить на контрольных файлах и заданном наборе изображений.